



SQL

SQL Basics to Advanced:

Complete Notes

1.Introduction

♦ What is a Database?

A **Database** is a collection of **organized data** that can be easily stored, managed, and retrieved.

👉 Example:

- A phonebook storing **Name** → **Phone Number**.
- A bank database storing **Customers, Accounts, Transactions**.

♦ DBMS vs RDBMS vs NoSQL

1. DBMS (Database Management System)

- Stores data in **files or simple tables**.
- Relationships between data are **not enforced** strictly.
- Example: **File System, MS Access**.

2. RDBMS (Relational DBMS)

- Stores data in **tables with rows & columns**.
- Follows **relations** ([like one-to-one, one-to-many](#)).
- Uses **SQL** to manage data.
- Example: **MySQL, PostgreSQL, Oracle, SQL Server**.

3. NoSQL

- Used for **big data, real-time apps, unstructured data**.
- Stores data in formats like **Key-Value, Documents, Graphs**.
- Example: **MongoDB (document-based), Redis (key-value), Neo4j (graph)**.

◆ Introduction to SQL

SQL (Structured Query Language) is used to interact with databases.

- You can **create tables, insert data, update, delete, and query**.

◆ Types of SQL Commands

1. DDL (Data Definition Language) – Defines structure /DDL commands are a subset of SQL used to define, modify, and manage the structure of database objects, rather than the data itself

- **CREATE** → create table
- **ALTER** → change table
- **DROP** → delete table
- **TRUNCATE** → delete all rows but keep structure

👉 Example:

```
CREATE TABLE Employee (  
    id INT,  
    name VARCHAR(50)  
);  
  
ALTER TABLE Employee ADD salary INT;  
  
DROP TABLE Employee;
```

2. DML (Data Manipulation Language) – Work with data/ manage and manipulate data within a database

- **INSERT** → add data
- **UPDATE** → modify data
- **DELETE** → remove data

👉 Example:

```
INSERT INTO Employee VALUES (1, 'Sunil', 50000);

UPDATE Employee SET salary = 60000 WHERE id = 1;

DELETE FROM Employee WHERE id = 1;
```

3. DQL (Data Query Language) – Query data /focuses on retrieving and querying data from a database

- **SELECT** → retrieve data

👉 Example:

```
SELECT * FROM Employee;
SELECT name, salary FROM Employee WHERE salary > 40000;
```

4. TCL (Transaction Control Language) – Control transactions /TCL commands are used to manage and control transactions within a database, ensuring data integrity and consistency.

- **COMMIT** → save changes
- **ROLLBACK** → undo changes
- **SAVEPOINT** → save temporary point

👉 Example:

```
BEGIN TRANSACTION;

UPDATE Employee SET salary = 70000 WHERE id = 1;

ROLLBACK; -- undo
COMMIT;   -- save permanently
```

5. DCL (Data Control Language) – Permissions & security / refers to a set of commands used to manage access permissions and control security within a database system.

- **GRANT** → give access
- **REVOKE** → remove access

👉 Example:

```
GRANT SELECT ON Employee TO user1;  
REVOKE SELECT ON Employee FROM user1;
```

◆ Database Objects

Tables → Store data in rows & columns

```
CREATE TABLE Products (id INT, name VARCHAR(50), price INT);
```

1. **Views** → Virtual table (saved query)

```
CREATE VIEW ExpensiveProducts AS  
SELECT name, price FROM Products WHERE price > 1000;
```

2. **Indexes** → Speed up searches

```
CREATE INDEX idx_name ON Products(name);
```

3. **Schemas** → Logical group of tables & objects / Think of it as a **folder inside a database**.

```
CREATE SCHEMA sales;
```

2 .SQL Basics

◆ 1. Creating Databases & Tables

Used to define where and how data will be stored.

👉 Example:

-- Create a database

```
CREATE DATABASE School;
```

-- Use database

```
USE School; #it use database school
```

-- Create a table

```
CREATE TABLE Students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    grade VARCHAR(5)  
);
```

◆ 2. Inserting Data (**INSERT INTO**)

Used to **add new records** into a table.

👉 Example:

```
INSERT INTO Students (id, name, age, grade)  
VALUES (1, 'Sunil', 22, 'A');
```

```
INSERT INTO Students VALUES (2, 'Riya', 20, 'B'); -- shortcut (if values  
for all columns)
```

◆ 3. Updating Data (**UPDATE**)

Used to **change existing records**.

👉 Example:

```
UPDATE Students
SET age = 23, grade = 'A+'
WHERE id = 1;
```

◆ 4. Deleting Data (**DELETE**)

Used to **remove records/row**.

👉 Example:

```
DELETE FROM Students WHERE id = 2;
```

◆ 5. Selecting Data (**SELECT**)

Used to **retrieve data**.

👉 Example:

```
SELECT * FROM Students;           -- all columns
SELECT name, grade FROM Students; -- specific columns
```

◆ 6. Filtering Data

We use conditions to get specific data.

(a) **WHERE** → filter rows

```
SELECT * FROM Students WHERE age > 20;
```

(b) **BETWEEN** → range filter

```
SELECT * FROM Students WHERE age BETWEEN 18 AND 22;
```

(c) **IN** → multiple values

```
SELECT * FROM Students WHERE grade IN ('A', 'B');
```

(d) **LIKE** → pattern search (% = any characters, _ = single character)

```
SELECT * FROM Students WHERE name LIKE 'S%'; -- names starting with S
SELECT * FROM Students WHERE name LIKE '_i%'; -- second letter is 'i'
```

(e) **IS NULL** → check empty values

```
SELECT * FROM Students WHERE grade IS NULL;
```

◆ 7. Sorting Data (**ORDER BY**)

Used to arrange results.

👉 Example:

```
SELECT * FROM Students ORDER BY age ASC; -- ascending
SELECT * FROM Students ORDER BY age DESC; -- descending
```

◆ 8. Limiting Data (**LIMIT, TOP**)

Used to restrict number of rows returned.

👉 Example:


```
-- MySQL / PostgreSQL
-- ➡ Returns the first 3 rows from the Students table.
SELECT * FROM Students LIMIT 3;

-- SQL Server
-- ➡ Returns the first 3 rows as well.
SELECT TOP 3 * FROM Students;
```

◆ 9. Aliases (AS)

Used to give **temporary names** to tables/columns.

👉 Example:

```
-- Column alias
SELECT name AS student_name, grade AS student_grade
FROM Students;

-- Table alias
SELECT s.name, s.grade
FROM Students AS s;
```

3 . SQL Functions

◆ 1. Aggregate Functions

Used to perform calculations on a **set of values** (return single value).

👉 Example (Students table with `id`, `name`, `age`, `grade`, `marks`):

```
SELECT COUNT(*) FROM Students;      -- total students
SELECT SUM(marks) FROM Students;    -- total marks
SELECT AVG(marks) FROM Students;    -- average marks
SELECT MIN(age) FROM Students;      -- youngest student
SELECT MAX(age) FROM Students;      -- oldest student
```

◆ 2. String Functions

Used to manipulate text (VARCHAR/CHAR).

👉 Examples:

```
SELECT CONCAT(name, ' - ', grade) AS student_info FROM Students;
-- Sunil - A

SELECT UPPER(name) FROM Students;    -- SUNIL
SELECT LOWER(name) FROM Students;    -- sunil

SELECT LTRIM('  Hello') AS left_trim; -- "Hello"
SELECT RTRIM('Hello  ') AS right_trim; -- "Hello"
SELECT TRIM('  Hello  ') AS full_trim;

SELECT SUBSTRING(name, 1, 3) FROM Students; -- First 3 letters- sun
SELECT REPLACE(name, 'i', 'ee') FROM Students; -- Replace 'i' with 'ee'

SELECT LENGTH(name) FROM Students;    -- number of characters in name
```

◆ 3. Date & Time Functions

Used to work with **dates and times**.

👉 Examples:

```
SELECT NOW();           -- Current date & time (2025-08-29 22:30:00)
SELECT CURDATE();       -- Current date only (2025-08-29)

SELECT DATEDIFF('2025-12-31', '2025-01-01'); -- 364 days
SELECT DATE_ADD(CURDATE(), INTERVAL 7 DAY);  -- today + 7 days

-- Extract part of a date
SELECT EXTRACT(YEAR FROM NOW()); -- 2025
SELECT EXTRACT(MONTH FROM NOW()); -- 8
SELECT EXTRACT(DAY FROM NOW()); -- 29
```

◆ 4. Mathematical Functions

Used to perform math operations.

👉 Examples:

```
SELECT ROUND(123.456, 2); -- 123.46 (rounded to 2 decimals)
SELECT CEIL(12.3);        -- 13 (next integer up)
SELECT FLOOR(12.9);       -- 12 (next integer down)
SELECT MOD(10, 3);        -- 1 (remainder)
```

◆ 5. Conversion Functions

Used to **convert data types**.

👉 Examples:

```
-- CAST: convert type
SELECT CAST('123' AS INT);      -- 123 (string to integer)

-- CONVERT: format/convert type
SELECT CONVERT('2025-08-29', DATE); -- output proper datatype not text.
```

4 . Filtering & Grouping

◆ 1. DISTINCT Keyword

Used to remove **duplicate values** from the result.

👉 Example (Students table: `id`, `name`, `age`, `grade`, `marks`)

```
SELECT grade FROM Students;      -- may return: A, A, B, B, C
SELECT DISTINCT grade FROM Students; -- returns: A, B, C
```

◆ 2. GROUP BY

Used to group rows having the **same values** into one group (often used with aggregate functions).

👉 Example:

```
-- Average marks of each grade
SELECT grade, AVG(marks) AS avg_marks
```

```
FROM Students
GROUP BY grade;
```

Output:

grade	avg_marks
A	85
B	70
C	60

◆ 3. HAVING

WHERE cannot be used with aggregate results, so we use **HAVING** with **GROUP BY**.

👉 Example:

```
-- Show grades where average marks > 75
SELECT grade, AVG(marks) AS avg_marks
FROM Students
GROUP BY grade
HAVING AVG(marks) > 75;
```

◆ 4. Multiple Aggregations

We can apply **more than one aggregate function** in the same query.

👉 Example:

```
-- For each grade: total students, avg marks, highest marks
SELECT grade,
       COUNT(*) AS total_students,
       AVG(marks) AS avg_marks,
       MAX(marks) AS highest_marks
```

```
FROM Students
GROUP BY grade;
```

Output:

grade	total_students	avg_marks	highest_marks
A	2	85	95
B	3	70	80
C	1	60	60

5 . SQL Joins & Relationships

◆ 1. Keys in SQL

Keys are used to **uniquely identify rows** and create **relationships** between tables.

✓ Types of Keys:

- **Primary Key** → Uniquely identifies each row. No duplicates, no NULL.

```
CREATE TABLE Students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT
);
```

- **Foreign Key** → Connects one table to another (reference).

```
CREATE TABLE Courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(50),
  student_id INT,
  FOREIGN KEY (student_id) REFERENCES Students(id)
);
```

- **Candidate Key** → Any column(s) that can be a primary key.
 👉 Example: In Students, both `id` and `email` can uniquely identify a student. Either can be chosen as primary.

-👉 So the **Candidate Keys** are:

`{id}, {email}, {phone}`

- **Composite Key** → Combination of 2 or more columns as a primary key.

```
CREATE TABLE Enrollments (
  student_id INT,
  course_id INT,
  PRIMARY KEY(student_id, course_id) -- composite key
);
```

♦ 2. Normalization Basics

Normalization = process of **organizing data** to reduce redundancy (duplicate data).

- **1NF (First Normal Form):** No repeating groups, atomic values only.
 👉 Example: Instead of storing multiple phone numbers in one column, create separate rows.
- **2NF (Second Normal Form):** Must be in 1NF + no partial dependency (non-key depends on part of composite key).
- **3NF (Third Normal Form):** Must be in 2NF + no transitive dependency (non-key depending on another non-key).

- **BCNF (Boyce-Codd Normal Form):** Stronger form of 3NF, eliminates more redundancy.

👉 Think: Higher normalization = less redundancy, but sometimes slower queries.

◆ 3. Joins in SQL

Joins are used to **combine data from multiple tables**.

Suppose we have:

Students Table

id	name
1	Sunil
2	Riya
3	Amit

Courses Table

course_id	course_name	student_id
101	Math	1
102	Science	2
103	History	4

✅ **INNER JOIN** → only matching rows

```
SELECT s.name, c.course_name
FROM Students s
INNER JOIN Courses c
ON s.id = c.student_id;
```


👉 Returns only (Sunil-Math, Riya-Science)

✅ **LEFT JOIN** → all rows from left, matching from right

```
SELECT s.name, c.course_name
FROM Students s
LEFT JOIN Courses c
ON s.id = c.student_id;
```

👉 Returns Sunil, Riya, Amit (Amit has NULL course).

✅ **RIGHT JOIN** → all rows from right, matching from left

```
SELECT s.name, c.course_name
FROM Students s
RIGHT JOIN Courses c
ON s.id = c.student_id;
```

👉 Returns Math, Science, History (History has NULL student).

✅ **FULL OUTER JOIN** → all rows from both tables

```
SELECT s.name, c.course_name
FROM Students s
FULL OUTER JOIN Courses c
ON s.id = c.student_id;
```

👉 Returns Sunil-Math, Riya-Science, Amit-NULL, NULL-History.

✅ **CROSS JOIN** → every combination

```
SELECT s.name, c.course_name
FROM Students s
CROSS JOIN Courses c;
```

👉 3 students × 3 courses = 9 combinations.

✅ SELF JOIN → join table with itself

-- Find students with same age

```
SELECT a.name AS student1, b.name AS student2
FROM Students a
JOIN Students b
ON a.age = b.age AND a.id <> b.id; -- not equal to
```

♦ 4. Subqueries (Nested Queries)

A query inside another query.

👉 Example:

-- Find students with marks > average marks

```
SELECT name
FROM Students
WHERE marks > (SELECT AVG(marks) FROM Students);
```

♦ 5. Correlated Subqueries

Unlike normal subqueries, correlated subqueries run **row by row**.

👉 Example:

-- Find students who have marks greater than avg marks of their grade

```
SELECT name, grade, marks
FROM Students s
WHERE marks > (
```

```
SELECT AVG(marks)
FROM Students
WHERE grade = s.grade --like 'A*' , 'A' , 'B' , - it select each all
);
```

6 . Advanced SQL Queries

◆ 1. Set Operators

Used to **combine results** from multiple queries.

(Tables must have same number of columns and compatible data types.)

👉 Example: Two tables — `Students_2024` and `Students_2025`.

- **UNION** → combines results, removes duplicates.

```
SELECT name FROM Students_2024
UNION
SELECT name FROM Students_2025;
```

- **UNION ALL** → combines results, keeps duplicates.

```
SELECT name FROM Students_2024
UNION ALL
SELECT name FROM Students_2025;
```

- **INTERSECT** → returns only common rows.

```
SELECT name FROM Students_2024
INTERSECT
SELECT name FROM Students_2025;
```

- **EXCEPT (or MINUS in Oracle)** → rows in first query, not in second.

```
SELECT name FROM Students_2024
EXCEPT
SELECT name FROM Students_2025;
```

◆ 2. CASE WHEN (Conditional Logic)

Like an **if-else** inside SQL.

👉 Example:

```
SELECT name,
       marks,
       CASE
         WHEN marks >= 90 THEN 'A'
         WHEN marks >= 75 THEN 'B'
         WHEN marks >= 60 THEN 'C'
         ELSE 'Fail'
       END AS grade
FROM Students;
```

- Alice → 95 → **>=90** → Grade **A**
 - Bob → 82 → **>=75** → Grade **B**
 - Carol → 67 → **>=60** → Grade **C**
 - David → 55 → **<60** → **Fail**
-

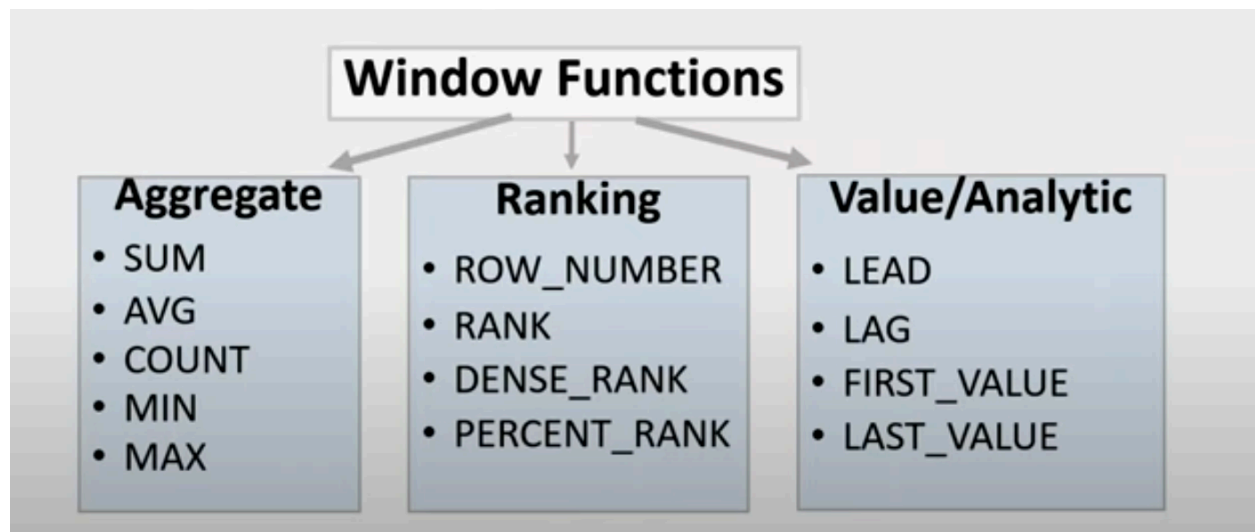
7 . window functions Advance SQL

(ROW_NUMBER, RANK, DENSE_RANK, NTILE, LEAD, LAG).

Window functions let you perform **calculations across a set of rows related to the current row**, without collapsing the rows into groups (like **GROUP BY** does).

👉 Syntax:

```
function() OVER (PARTITION BY column ORDER BY column)
```



◆ **1. Ranking Functions : Used for ordering and ranking rows.**

Function	Description	Example
ROW_NUMBER()	Unique number for each row (no ties).	<code>ROW_NUMBER() OVER (ORDER BY salary DESC)</code>
RANK()	Ranking with gaps if ties.	<code>RANK() OVER (ORDER BY salary DESC)</code>
DENSE_RANK()	Ranking without gaps.	<code>DENSE_RANK() OVER (ORDER BY salary DESC)</code>
NTILE(n)	Divide rows into <i>n</i> equal groups (buckets).	<code>NTILE(4) OVER (ORDER BY marks DESC)</code>

📌 Example Table: **Employees:**

id	name	salary	marks
1	Alice	5000	95
2	Bob	7000	88
3	Carol	7000	76

4	David	4000	60
5	Eva	4000	55

1. ROW_NUMBER() → unique number for each row (no ties)

```
SELECT name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM Employees;
```

name	salary	row_num
Bob	7000	1
Carol	7000	2
Alice	5000	3
David	4000	4
Eva	4000	5

✓ Even if salaries tie, row numbers are **unique**.

2. RANK() → ranking with gaps if ties

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rank_pos  
FROM Employees;
```

name	salary	rank_pos
Bob	7000	1
Carol	7000	1
Alice	5000	3
David	4000	4
Eva	4000	4

✓ Bob & Carol tie → both rank 1, next rank skips to **3**.

3. DENSE_RANK() → ranking without gaps

```
SELECT name, salary,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_pos  
FROM Employees;
```

name	salary	dense_rank_pos
Bob	7000	1
Carol	7000	1
Alice	5000	2
David	4000	3
Eva	4000	3

✓ Bob & Carol tie → both rank 1, next rank is **2** (no gap).

4. NTILE(n) → divide rows into n groups

```
SELECT name, marks,  
       NTILE(4) OVER (ORDER BY marks DESC) AS quartile  
FROM Employees;
```

name	marks	quartile
Alice	95	1
Bob	88	1
Carol	76	2
David	60	3
Eva	55	4

✓ Data split into **4 groups** (quartiles) based on **marks**.

◆ 2. Aggregate Functions as Window Functions

Normal aggregates, but used **without collapsing rows**.

Function	Description	Example
SUM()	Running total / sum over partition.	<code>SUM(sales) OVER (PARTITION BY region ORDER BY date)</code>
AVG()	Running / moving average.	<code>AVG(salary) OVER (ORDER BY hire_date)</code>
MIN()	Minimum value in window.	<code>MIN(marks) OVER (PARTITION BY grade)</code>
MAX()	Maximum value in window.	<code>MAX(marks) OVER (PARTITION BY grade)</code>
COUNT()	Row count in window.	<code>COUNT(*) OVER (PARTITION BY dept)</code>

Examples :

1. SUM() – Running Total

👉 Calculates cumulative total.

```
SELECT
    region,
    date,
    sales,
    SUM(sales) OVER (PARTITION BY region ORDER BY date) AS running_total
FROM sales_table;
```

PARTITION BY region :

→ SQL calculates totals separately for each region (East and West).

ORDER BY date :

→ Within each region, sales are added in date order.

SUM(sales) OVER(...): cumulative sum

→ Adds up sales one row at a time → creates a running total.

📌 Example Output:

region	date	sales	running_total
East	2023-01-01	100	100
East	2023-01-02	200	300
East	2023-01-03	150	450

2. AVG() – Moving Average

👉 Shows average salary up to that point (by hiring order).

```
SELECT
    emp_name,
    hire_date,
    salary,
    AVG(salary) OVER (ORDER BY hire_date) AS running_avg
FROM employees;
```

📌 Example Output: **cumulative avg.**

emp_name	hire_date	salary	running_avg
John	2020-01-01	3000	3000
Mary	2020-02-01	4000	3500
Alex	2020-03-01	5000	4000

3. MIN() – Minimum Value in Window

👉 Finds lowest marks in each grade.

```
SELECT
    student_name,
    grade,
    marks,
    MIN(marks) OVER (PARTITION BY grade) AS min_marks_in_grade
FROM students;
```

📌 Example Output:

student_name	grade	marks	min_marks_in_grade
Aditi	A	90	85
Rahul	A	85	85
Sneha	B	70	60
Mohit	B	60	60

4. MAX() – Maximum Value in Window

👉 Finds highest marks in each grade.

```
SELECT
    student_name,
    grade,
    marks,
    MAX(marks) OVER (PARTITION BY grade) AS max_marks_in_grade
FROM students;
```

📌 Example Output:

student_name	grade	marks	max_marks_in_grade
Aditi	A	90	95
Rahul	A	85	95
Sneha	B	70	80
Mohit	B	80	80

5. COUNT() – Row Count

👉 Counts how many employees in each department.

```
SELECT
    emp_name,
    dept,
    COUNT(*) OVER (PARTITION BY dept) AS dept_count
FROM employees;
```

📌 Example Output:

emp_name	dept	dept_count
John	HR	3
Mary	HR	3
Alex	HR	3
Sneha	IT	2
Mohit	IT	2

◆ 3. Value Functions (Navigation Functions)

Used to look at values **before/after/current** row.

emp_id	emp_name	hire_date	salary
1	Asha	2020-01-01	3000
2	Ravi	2020-02-01	3500
3	Neha	2020-03-01	4000
4	Karan	2020-04-01	4500
5	Meera	2020-05-01	5000

Function	Description	Example
LEAD(expr, offset, default)	Value from <i>next</i> row.	<code>LEAD(salary, 1) OVER (ORDER BY hire_date)</code>
LAG(expr, offset, default)	Value from <i>previous</i> row.	<code>LAG(salary, 1) OVER (ORDER BY hire_date)</code>
FIRST_VALUE(expr)	First value in the window.	<code>FIRST_VALUE(salary) OVER (ORDER BY hire_date)</code>
LAST_VALUE(expr)	Last value in the window.	<code>LAST_VALUE(salary) OVER (ORDER BY hire_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)</code>
NTH_VALUE(expr, n)	Nth value in the window.	<code>NTH_VALUE(salary, 3) OVER (ORDER BY hire_date)</code>

Example :

1. LEAD() → Get next row value

```
SELECT
    emp_name,
    salary,
    LEAD(salary, 1) OVER (ORDER BY hire_date) AS next_salary
FROM employees;
```

Output

emp_name	salary	next_salary
Asha	3000	3500
Ravi	3500	4000
Neha	4000	4500
		like this

2. LAG() → Get previous row value

```
SELECT
    emp_name,
    salary,
    LAG(salary, 1) OVER (ORDER BY hire_date) AS prev_salary
FROM employees;
```

Output

emp_name	salary	prev_salary
Asha	3000	NULL
Ravi	3500	3000
Neha	4000	3500
Karan	4500	4000
Meera	5000	4500

3. FIRST_VALUE() → Get first row in window

```
SELECT
    emp_name,
    salary,
    FIRST_VALUE(salary) OVER (ORDER BY hire_date) AS first_salary
FROM employees;
```

Output

emp_name	salary	first_salary
Asha	3000	3000
Ravi	3500	3000
Neha	4000	3000
Karan	4500	3000
Meera	5000	3000

4. LAST_VALUE() → Get last row in window

(Needs frame clause to look ahead at all rows)

```
SELECT
    emp_name,
    salary,
    LAST_VALUE(salary) OVER (
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_salary
FROM employees;
```

Output

emp_name	salary	last_salary
Asha	3000	5000
Ravi	3500	5000
Neha	4000	5000
Karan	4500	5000
Meera	5000	5000

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING

By default, **LAST_VALUE** only looks at the **current window up to current row**, not the true last row.

👉 To fix that, we extend the window to include **all rows from the beginning (UNBOUNDED PRECEDING) to the end (UNBOUNDED FOLLOWING)**.

✅ **last_salary** column always shows **David's salary (5000)** because he's the most recently hired.

5. NTH_VALUE() → Get Nth row in window

```
SELECT
    emp_name,
    salary,
    NTH_VALUE(salary, 3) OVER (
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS third_salary
FROM employees;
```

Output

emp_name	salary	third_salary
Asha	3000	4000
Ravi	3500	4000
Neha	4000	4000
Karan	4500	4000
Meera	5000	4000

◆ 4. Statistical / Distribution Functions

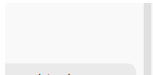
Useful for analytics.

Function	Description	Example
CUME_DIST()	Cumulative distribution (percent ≤ current row).	<code>CUME_DIST() OVER (ORDER BY marks)</code>
PERCENT_RANK()	Relative rank between 0 and 1.	<code>PERCENT_RANK() OVER (ORDER BY salary)</code>

1. CUME_DIST()

👉 Returns the **cumulative distribution**: the proportion of rows with a value **less than or equal to** the current row.

Formula:



$$CUME_DIST(x) = \frac{\text{Number of rows with value} \leq x}{\text{Total number of rows}}$$

Example:

```
SELECT
    student_name,
    marks,
    CUME_DIST() OVER (ORDER BY marks) AS cume_dist
FROM students;
```

Suppose **students** table:

student_name	marks
A	40
B	50
C	60
D	80

 Calculation:

- For marks = 40 $\rightarrow 1/4 = 0.25$
- For marks = 50 $\rightarrow 2/4 = 0.50$
- For marks = 60 $\rightarrow 3/4 = 0.75$
- For marks = 80 $\rightarrow 4/4 = 1.00$

👉 Means: student scoring 60 is at the **75th percentile** (better than or equal to 75% students).

2. PERCENT_RANK()

👉 Returns the **relative rank** of a row **between 0 and 1**.

Formula:

$$\text{PERCENT_RANK}(x) = \frac{\text{Rank of row} - 1}{\text{Total rows} - 1}$$

Example:

```
SELECT
    emp_name,
    salary,
    PERCENT_RANK() OVER (ORDER BY salary) AS percent_rank
FROM employees;
```

Suppose **employees** table:

emp_name	salary
A	1000
B	2000
C	3000
D	5000

 Calculation:

- Rank of A = 1 $\rightarrow (1-1)/(4-1) = 0.00$
- Rank of B = 2 $\rightarrow (2-1)/(3) = 0.33$
- Rank of C = 3 $\rightarrow (3-1)/(3) = 0.67$
- Rank of D = 4 $\rightarrow (4-1)/(3) = 1.00$

👉 Means: an employee with salary 3000 has a **percent rank of 0.67** (i.e. 67% position in salary list).

✅ Difference:

- `CUME_DIST()` = % of rows $\leq$ **current value**.
 - `PERCENT_RANK()` = relative rank **based on ordering** (always starts at 0 and ends at 1).
-

◆ 5. Framing (Window Frames)

You can control **which rows the window looks at**.

- Default: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
- Examples:

```
-- Running total
SUM(sales) OVER (ORDER BY date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

👉 Adds up sales **from the first row till the current row** (cumulative sum).

- **UNBOUNDED PRECEDING** $\rightarrow$ start from very first row
- **CURRENT ROW** $\rightarrow$ end at current row

date	sales	running_total
1-Jan	100	100
2-Jan	200	300
3-Jan	150	450
4-Jan	50	500

```
-- Moving average of last 3 rows
AVG(sales) OVER (ORDER BY date
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```

👉 Averages the sales of **current row + 2 rows before it**.

- This is a **sliding window** (like a moving average in time-series).

date	sales	moving_avg(last 3)
1-Jan	100	100.0 (only itself)
2-Jan	200	150.0 (100,200)
3-Jan	150	150.0 (100,200,150)
4-Jan	50	133.3 (200,150,50)
5-Jan	300	166.7 (150,50,300)

◆ 4. CTE (Common Table Expressions)

Temporary result set that can be reused in query.

👉 Example:

```
WITH HighScorers AS (  
    SELECT name, marks  
    FROM Students  
    WHERE marks > 80  
)  
SELECT * FROM HighScorers;
```

WITH HighScorers AS (...)

- Here, we are creating a **temporary result set** (like a virtual table) called **HighScorers**.
- Inside it, we select only those students from **Students** table who have **marks > 80**.

name	marks
Aditi	95
Rahul	76
Neha	88
Sameer	65

- Then **HighScorers** will temporarily hold:

name	marks
Aditi	95
Neha	88

```
SELECT * FROM HighScorers;
```

- Now, instead of querying the main **Students** table again, we query this **temporary CTE**.
- Output will be:

name	marks
------	-------

Aditi	95
-------	----

Neha	88
------	----

◆ 5. Recursive Queries (Recursive CTE)

Used for hierarchical data (org charts, family trees, category-subcategory).

 **Example Table (**Employees**)**

id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2
5	Emma	2

Query Explanation

```
WITH RECURSIVE EmployeeHierarchy AS (  
  -- 1. Base case (find the top boss)  
  SELECT id, name, manager_id  
  FROM Employees
```

```
WHERE manager_id IS NULL -- Alice (the CEO)

UNION ALL

-- 2. Recursive step (find employees reporting to the previous level)
SELECT e.id, e.name, e.manager_id
FROM Employees e
JOIN EmployeeHierarchy eh
ON e.manager_id = eh.id
)
-- 3. Final output
SELECT * FROM EmployeeHierarchy;
```

How It Works

1. **Base case:** Start with `manager_id IS NULL` → Alice (CEO).
2. **Recursive step:** Find all employees whose `manager_id = Alice.id` → Bob, Charlie.
3. Repeat:
 - Employees under Bob → David, Emma.
 - Employees under Charlie → (none).
4. Stops when no new rows are found.

Final Output

id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2
5	Emma	2

◆ 8. Transactions & Security

1. Transactions

👉 A **Transaction** is a group of SQL statements that execute together as one unit.
If everything succeeds → **COMMIT** (save).
If something fails → **ROLLBACK** (undo).

Example:

```
BEGIN;    -- start transaction

UPDATE Accounts SET balance = balance - 500 WHERE id = 1; -- deduct from A
UPDATE Accounts SET balance = balance + 500 WHERE id = 2; -- add to B

COMMIT;   -- save both together
```

✅ If second statement fails, use:

```
ROLLBACK; -- undo everything
```

ROLLBACK can undo all the changes made **after a BEGIN** and **before a COMMIT**.

This ensures **data consistency** (money doesn't disappear or duplicate).

2. ACID Properties

Transactions follow **ACID** rules:

- **Atomic** → All or nothing (COMMIT/ROLLBACK).
- **Consistent** → Database remains valid.
- **Isolated** → Transactions don't interfere with each other.
- **Durable** → Once committed, changes are permanent.

3. Savepoint

👉 A **Savepoint** is a checkpoint inside a transaction.

Example:

```
BEGIN;  
  
UPDATE Accounts SET balance = balance - 200 WHERE id = 1;  
SAVEPOINT deduct_done;  
  
UPDATE Accounts SET balance = balance + 200 WHERE id = 2;  
  
ROLLBACK TO deduct_done; -- undo only from savepoint  
COMMIT;
```

BEGIN → starts a new transaction.

UPDATE (id=1) → deducts ₹200 from account 1.

SAVEPOINT deduct_done → marks a point to which we can roll back.

UPDATE (id=2) → tries to add ₹200 to account 2.

ROLLBACK TO deduct_done → cancels only the update after the savepoint (addition to account 2).

COMMIT → permanently saves the deduction from account 1 (before the savepoint).

4. Security – GRANT & REVOKE

👉 Used to **control user permissions**.

- **GRANT** → Give access
- **REVOKE** → Remove access

Example:

```
GRANT SELECT, INSERT ON Students TO user1;    -- give read & insert rights
REVOKE INSERT ON Students FROM user1;         -- remove insert rights
```

Gives **user1** permission to **read (SELECT)** and **add (INSERT)** records in the **Students** table.
Removes the **insert permission** from **user1**, so now they can only **read** the table.

5. Roles (group of permissions)

👉 Instead of giving permissions one by one, create a role.

```
CREATE ROLE teacher;
GRANT SELECT, UPDATE ON Students TO teacher;
GRANT teacher TO user1;
```

Now **user1** has teacher permissions.

- Makes a role called **teacher**. Think of it as a **group of permissions**.
- The **teacher** role can **read (SELECT)** and **modify (UPDATE)** the **Students** table.
- Now **user1** automatically has **all permissions of the teacher** role.

✅ **Result:**

- **user1** can **view** and **update** student records.
 - If more users are added to the **teacher** role later, they automatically get the same permissions.
-

◆ 9 . Pivot & Unpivot Queries

1. Pivot – Transform rows → columns

Sample **SalesData** table:

year	quarter	sales
2025	Q1	100
2025	Q2	150
2025	Q3	200
2025	Q4	250

Pivot Query:

```
SELECT *  
FROM (SELECT year, quarter, sales FROM SalesData) AS src  
PIVOT (  
    SUM(sales) FOR quarter IN ([Q1], [Q2], [Q3], [Q4])  
) AS pvt;
```

- Inner query:

SELECT year, quarter, sales FROM SalesData

Gets the raw data (year, quarter, sales).

- PIVOT:

**PIVOT (SUM(sales) FOR quarter IN ([Q1], [Q2], [Q3], [Q4]))
AS pvt**

- Turns the **quarter** values (Q1–Q4) into columns.
- Calculates **SUM(sales)** for each quarter.

✓ **Result: One row per year, with columns Q1, Q2, Q3, Q4 showing total sales.**

Output:

year	Q1	Q2	Q3	Q4
2025	100	150	200	250

2. Unpivot – Transform columns → rows

Suppose we have a pivoted table:

year	Q1	Q2	Q3	Q4
2025	100	150	200	250

Unpivot Query:

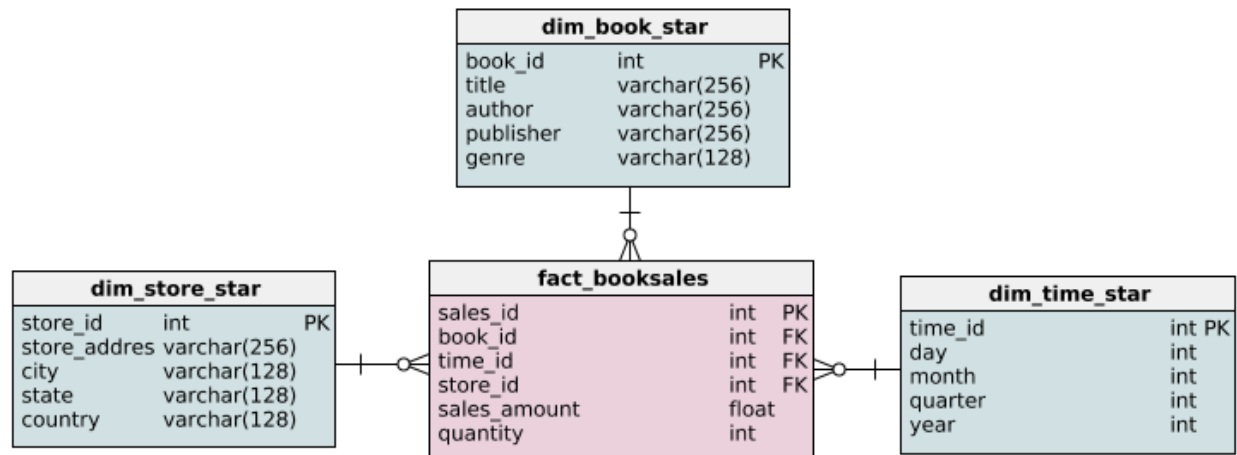
```
SELECT year, quarter, sales
FROM SalesData
UNPIVOT (
    sales FOR quarter IN ([Q1], [Q2], [Q3], [Q4])
) AS unpvt;
```

Output:

year	quarter	sales
2025	Q1	100
2025	Q2	150
2025	Q3	200
2025	Q4	250

✓ **Explanation:**

- Converts **columns Q1–Q4 back into rows**.
- Useful for analysis where you need data in **long format** instead of wide format.



1. Show book title and sales amount for each sale.

```

SELECT b.title, f.sales_amount
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id;
  
```

2. Show store city and total quantity sold in that store.

```

SELECT s.city, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
GROUP BY s.city;
  
```

3. List all sales with book title, author, and quantity.

```

SELECT b.title, b.author, f.quantity
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id;
  
```

4. Show store address and the books sold there.

```

SELECT s.store_address, b.title
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
JOIN dim_book_star b ON f.book_id = b.book_id;
  
```

5. Display month, year, book title, and sales amount.

```
SELECT t.month, t.year, b.title, f.sales_amount
FROM fact_booksales f
JOIN dim_time_star t ON f.time_id = t.time_id
JOIN dim_book_star b ON f.book_id = b.book_id;
```

6. Show all authors and in which stores their books sold.

```
SELECT b.author, s.store_address
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
JOIN dim_store_star s ON f.store_id = s.store_id;
```

7. List each store and total sales amount for that store.

```
SELECT s.store_address, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
GROUP BY s.store_address;
```

8. Show book title and the city where it was sold.

```
SELECT b.title, s.city
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
JOIN dim_store_star s ON f.store_id = s.store_id;
```

9. Show store country and sales amount of each sale.

```
SELECT s.country, f.sales_amount
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id;
```

10. List all books and the number of stores they were sold in.

```
SELECT b.title, COUNT(DISTINCT f.store_id) AS store_count
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.title;
```

11. Display genre and total quantity sold for that genre.

```
SELECT b.genre, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.genre;
```

12. Show publisher and all cities where their books sold.

```
SELECT DISTINCT b.publisher, s.city
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
JOIN dim_store_star s ON f.store_id = s.store_id;
```

13. Show total sales amount per year with book titles included.

```
SELECT t.year, b.title, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_time_star t ON f.time_id = t.time_id
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY t.year, b.title;
```

14. List year, quarter, book title, and sales quantity.

```
SELECT t.year, t.quarter, b.title, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_time_star t ON f.time_id = t.time_id
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY t.year, t.quarter, b.title;
```

15. Show each store along with all books sold and their authors.

```
SELECT s.store_address, b.title, b.author
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
JOIN dim_book_star b ON f.book_id = b.book_id;
```

16. Find total sales amount across all stores.

```
SELECT SUM(sales_amount) AS total_sales
FROM fact_booksales;
```

17. Find total quantity sold for each book.

```
SELECT book_id, SUM(quantity) AS total_qty
FROM fact_booksales
GROUP BY book_id;
```

18. Compute total sales amount for each year.

```
SELECT t.year, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_time_star t ON f.time_id = t.time_id
GROUP BY t.year;
```

19. Compute total sales amount per month.

```
SELECT t.month, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_time_star t ON f.time_id = t.time_id
GROUP BY t.month;
```

20. Find average sales amount per sale.

```
SELECT AVG(sales_amount) AS avg_sales
FROM fact_booksales;
```

21. Find max sales amount recorded in any sale.

```
SELECT MAX(sales_amount) AS max_sale
FROM fact_booksales;
```

22. Count total number of sales transactions.

```
SELECT COUNT(*) AS total_transactions
FROM fact_booksales;
```


23. Find number of sales done per store.

```
SELECT store_id, COUNT(*) AS total_sales
FROM fact_booksales
GROUP BY store_id;
```

24. Compute total quantity sold for each genre.

```
SELECT b.genre, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.genre;
```

25. Compute total sales amount for each author.

```
SELECT b.author, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.author;
```

26. Show top 5 books by sales amount.

```
SELECT b.title, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.title
ORDER BY total_sales DESC
LIMIT 5;
```

27. Show bottom 5 stores by quantity sold.

```
SELECT s.store_address, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
GROUP BY s.store_address
ORDER BY total_qty ASC
LIMIT 5;
```

28. Compute total sales by country.

```
SELECT s.country, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
GROUP BY s.country;
```

29. Compute monthly quantity sold per store.

```
SELECT s.store_address, t.month, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
JOIN dim_time_star t ON f.time_id = t.time_id
GROUP BY s.store_address, t.month;
```

30. Find total sales for each publisher.

```
SELECT b.publisher, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.publisher;
```

31. Find books whose total quantity sold is above the average quantity.

```
SELECT book_id, SUM(quantity) AS total_qty
FROM fact_booksales
GROUP BY book_id
HAVING SUM(quantity) >
    (SELECT AVG(quantity) FROM fact_booksales);
```

32. Find stores whose total sales amount is above average store sales amount.

```
SELECT store_id, SUM(sales_amount) AS total_sales
FROM fact_booksales
GROUP BY store_id
HAVING SUM(sales_amount) >
    (SELECT AVG(total_sales) FROM (
        SELECT SUM(sales_amount) AS total_sales
        FROM fact_booksales
        GROUP BY store_id) x);
```

33. Find books not sold in the year 2023.

```
SELECT *
FROM dim_book_star
WHERE book_id NOT IN (
    SELECT f.book_id
    FROM fact_booksales f
    JOIN dim_time_star t ON f.time_id = t.time_id
    WHERE t.year = 2023
);
```

34. Find authors with total sales above average author sales.

```
SELECT b.author, SUM(f.sales_amount) AS total_sales
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.author
HAVING SUM(f.sales_amount) >
    (SELECT AVG(author_total) FROM (
        SELECT SUM(f2.sales_amount) AS author_total
        FROM fact_booksales f2
        JOIN dim_book_star b2 ON f2.book_id = b2.book_id
        GROUP BY b2.author
    ) a);
```

35. Find stores in countries with total sales > 1,000,000.

```
SELECT *
FROM dim_store_star
WHERE country IN (
    SELECT s.country
    FROM fact_booksales f
    JOIN dim_store_star s ON f.store_id = s.store_id
    GROUP BY s.country
    HAVING SUM(f.sales_amount) > 1000000
);
```

36. Find books sold in more than 3 different cities.

```
SELECT b.title
FROM fact_booksales f
JOIN dim_store_star s ON f.store_id = s.store_id
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.title
HAVING COUNT(DISTINCT s.city) > 3;
```

37. Find publishers with any book having above-average sales_amount.

```
SELECT DISTINCT b.publisher
FROM dim_book_star b
JOIN fact_booksales f ON b.book_id = f.book_id
WHERE f.sales_amount > (SELECT AVG(sales_amount) FROM fact_booksales);
```

38. Find genres with total quantity sold > average genre quantity.

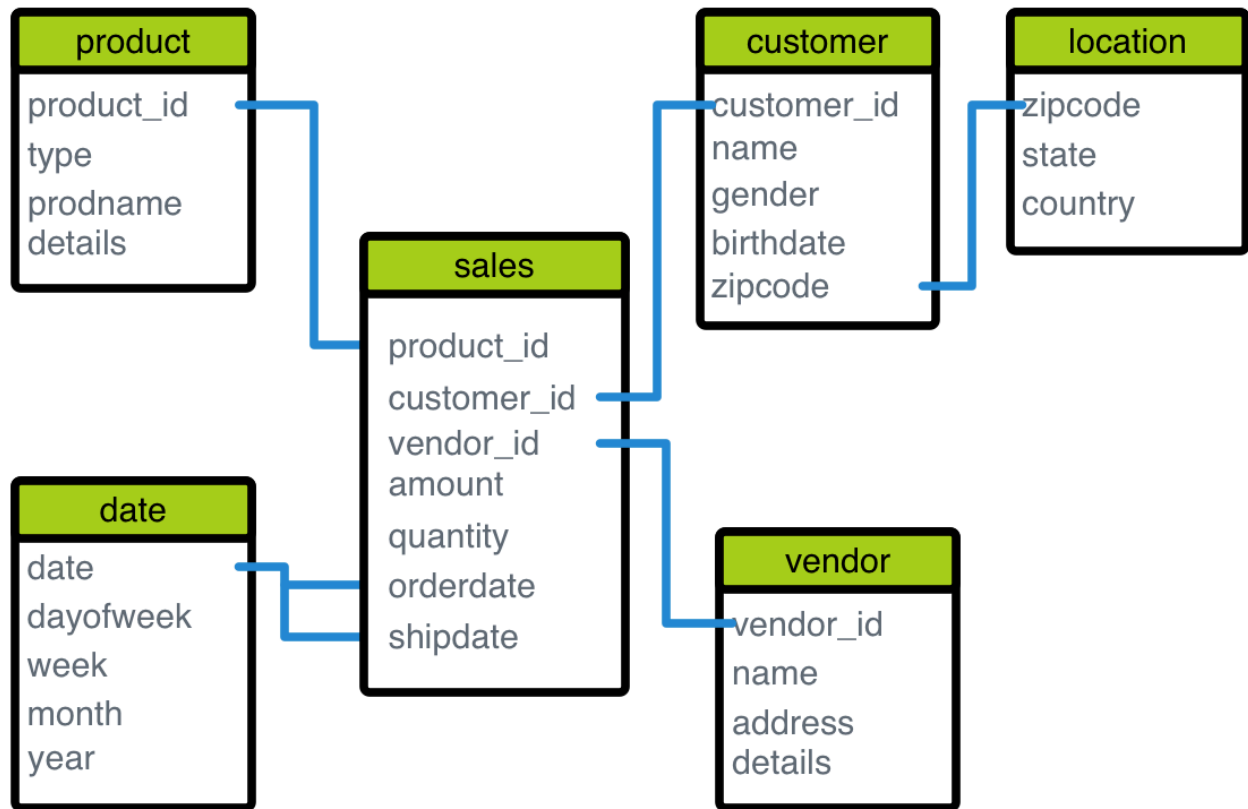
```
SELECT b.genre, SUM(f.quantity) AS total_qty
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.genre
HAVING SUM(f.quantity) >
    (SELECT AVG(qty) FROM (
        SELECT SUM(f2.quantity) AS qty
        FROM fact_booksales f2
        JOIN dim_book_star b2 ON f2.book_id = b2.book_id
        GROUP BY b2.genre
    ) x);
```

39. Find books sold only in one store.

```
SELECT b.title
FROM fact_booksales f
JOIN dim_book_star b ON f.book_id = b.book_id
GROUP BY b.title
HAVING COUNT(DISTINCT f.store_id) = 1;
```

40. Find stores that sold more books (quantity) than average store quantity.

```
SELECT store_id, SUM(quantity) AS total_qty
FROM fact_booksales
GROUP BY store_id
HAVING SUM(quantity) >
    (SELECT AVG(qty) FROM (
        SELECT SUM(quantity) AS qty
        FROM fact_booksales
        GROUP BY store_id
    ) x);
```



1. List all customers.

```
SELECT * FROM customer;
```

2. List all products.

```
SELECT * FROM product;
```

3. List all vendors.

```
SELECT * FROM vendor;
```

4. Show customer names and gender.

```
SELECT name, gender FROM customer;
```

5. Show all product names.

```
SELECT prodname FROM product;
```

6. Show distinct product types.

```
SELECT DISTINCT type FROM product;
```

7. Show all sales records.

```
SELECT * FROM sales;
```

8. Find customers in zipcode '560001'.

```
SELECT * FROM customer WHERE zipcode = '560001';
```

9. Products of type 'Electronics'.

```
SELECT * FROM product WHERE type = 'Electronics';
```

10. Show sales where quantity > 5.

```
SELECT * FROM sales WHERE quantity > 5;
```

11. Show customer name and order amount.

```
SELECT c.name, s.amount  
FROM customer c  
JOIN sales s ON c.customer_id = s.customer_id;
```

12. Total quantity sold per product.

```
SELECT p.prodname, SUM(s.quantity) AS total_qty  
FROM product p  
JOIN sales s ON p.product_id = s.product_id  
GROUP BY p.prodname;
```

13. Show vendor names with products they sold.

```
SELECT v.name, p.prodname  
FROM vendor v  
JOIN sales s ON v.vendor_id = s.vendor_id  
JOIN product p ON s.product_id = p.product_id;
```

14. Show customer & vendor involved in each sale.

```
SELECT c.name AS customer, v.name AS vendor
FROM sales s
JOIN customer c ON s.customer_id = c.customer_id
JOIN vendor v ON s.vendor_id = v.vendor_id;
```

15. Show customer name and country.

```
SELECT c.name, l.country
FROM customer c
JOIN location l ON c.zipcode = l.zipcode;
```

16. Show product name and order date.

```
SELECT p.prodname, s.orderdate
FROM sales s
JOIN product p ON s.product_id = p.product_id;
```

17. Show customer, product and amount.

```
SELECT c.name, p.prodname, s.amount
FROM sales s
JOIN customer c ON s.customer_id = c.customer_id
JOIN product p ON s.product_id = p.product_id;
```

18. Count total customers.

```
SELECT COUNT(*) AS total_customers FROM customer;
```

19. Count total products.

```
SELECT COUNT(*) AS total_products FROM product;
```

20. Total sales amount.

```
SELECT SUM(amount) AS total_sales FROM sales;
```


21. Number of sales made by each customer.

```
SELECT customer_id, COUNT(*) AS total_sales
FROM sales
GROUP BY customer_id;
```

22. Number of sales for each product.

```
SELECT product_id, COUNT(*) AS total_sales
FROM sales
GROUP BY product_id;
```

23. Total quantity sold by each vendor.

```
SELECT vendor_id, SUM(quantity) AS total_qty
FROM sales
GROUP BY vendor_id;
```

24. Total sales amount by product type.

```
SELECT p.type, SUM(s.amount) AS total_amount
FROM sales s
JOIN product p ON s.product_id = p.product_id
GROUP BY p.type;
```

25. Average order amount.

```
SELECT AVG(amount) AS avg_amount FROM sales;
```

26. Products whose revenue > 10,000.

```
SELECT p.prodname, SUM(s.amount) AS revenue
FROM sales s
JOIN product p ON s.product_id = p.product_id
GROUP BY p.prodname
HAVING SUM(s.amount) > 10000;
```

27. Customers with more than 5 orders.

```
SELECT customer_id, COUNT(*) AS orders
FROM sales
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

28. Vendors selling > 20 units total.

```
SELECT vendor_id, SUM(quantity) AS total_units
FROM sales
GROUP BY vendor_id
HAVING SUM(quantity) > 20;
```

29. Products never sold.

```
SELECT *
FROM product p
LEFT JOIN sales s ON p.product_id = s.product_id
WHERE s.product_id IS NULL;
```

30. Customers who never placed orders.

```
SELECT *
FROM customer c
LEFT JOIN sales s ON c.customer_id = s.customer_id
WHERE s.customer_id IS NULL;
```

31. List customers with their state.

```
SELECT c.name, l.state
FROM customer c
JOIN location l ON c.zipcode = l.zipcode;
```

32. Customer count by country.

```
SELECT l.country, COUNT(*) AS total_customers
FROM customer c
JOIN location l ON c.zipcode = l.zipcode
GROUP BY l.country;
```

33. Sales done in year 2024.

```
SELECT *
FROM sales
WHERE YEAR(orderdate) = 2024;
```

34. Products sold in January.

```
SELECT p.prodname
FROM sales s
JOIN product p ON s.product_id = p.product_id
WHERE MONTH(orderdate) = 1;
```

35. Show customer, orderdate, and day of week.

```
SELECT c.name, s.orderdate, d.dayofweek
FROM sales s
JOIN date d ON s.orderdate = d.date
JOIN customer c ON s.customer_id = c.customer_id;
```

36. Customers older than 30.

```
SELECT *
FROM customer
WHERE TIMESTAMPDIFF(YEAR, birthdate, CURDATE()) > 30;
```

37. Top 5 products by quantity sold.

```
SELECT p.prodname, SUM(s.quantity) AS total_qty
FROM sales s
JOIN product p ON s.product_id = p.product_id
GROUP BY p.prodname
ORDER BY total_qty DESC
LIMIT 5;
```

38. Total spending by customer.

```
SELECT c.name, SUM(s.amount) AS total_spent
FROM sales s
JOIN customer c ON s.customer_id = c.customer_id
GROUP BY c.name;
```

39. Number of customers served by each vendor.

```
SELECT vendor_id, COUNT(DISTINCT customer_id) AS total_customers
FROM sales
GROUP BY vendor_id;
```

40. Orders where shipdate > orderdate.

```
SELECT *  
FROM sales  
WHERE shipdate > orderdate;
```

41. Product with highest revenue.

```
SELECT p.prodname, SUM(s.amount) AS revenue  
FROM sales s  
JOIN product p ON s.product_id = p.product_id  
GROUP BY p.prodname  
ORDER BY revenue DESC  
LIMIT 1;
```

42. Vendor with highest sales revenue.

```
SELECT v.name, SUM(s.amount) AS revenue  
FROM sales s  
JOIN vendor v ON s.vendor_id = v.vendor_id  
GROUP BY v.name  
ORDER BY revenue DESC  
LIMIT 1;
```

43. Customers who bought ALL product types.

```
SELECT customer_id  
FROM sales s  
JOIN product p ON s.product_id = p.product_id  
GROUP BY customer_id  
HAVING COUNT(DISTINCT p.type) = (  
    SELECT COUNT(DISTINCT type) FROM product  
);
```

44. Vendors who sold to ALL customers.

```
SELECT vendor_id  
FROM sales  
GROUP BY vendor_id  
HAVING COUNT(DISTINCT customer_id) = (  
    SELECT COUNT(*) FROM customer  
);
```

45. Customers who bought more than 3 different products.

```
SELECT customer_id, COUNT(DISTINCT product_id) AS product_count
FROM sales
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) > 3;
```

46. Products with total quantity above average (subquery).

```
SELECT product_id, SUM(quantity) AS total_qty
FROM sales
GROUP BY product_id
HAVING SUM(quantity) > (
    SELECT AVG(q) FROM (
        SELECT SUM(quantity) AS q
        FROM sales
        GROUP BY product_id
    ) AS x
);
```

47. Customers whose total spending > average spending.

```
SELECT customer_id, SUM(amount) AS total_spending
FROM sales
GROUP BY customer_id
HAVING SUM(amount) > (
    SELECT AVG(total)
    FROM (SELECT SUM(amount) AS total FROM sales GROUP BY customer_id) t
);
```

48. Products bought ONLY by female customers.

```
SELECT p.prodname
FROM product p
WHERE p.product_id IN (
    SELECT product_id
    FROM sales s
    JOIN customer c ON s.customer_id = c.customer_id
    GROUP BY product_id
    HAVING COUNT(DISTINCT gender) = 1
    AND MIN(gender) = 'F'
);
```

49. Vendors below average vendor revenue.

```
SELECT vendor_id, SUM(amount) AS revenue
FROM sales
GROUP BY vendor_id
HAVING SUM(amount) < (
    SELECT AVG(r)
    FROM (SELECT SUM(amount) AS r FROM sales GROUP BY vendor_id) x
);
```

50. Customers older than the average age.

```
SELECT *
FROM customer
WHERE TIMESTAMPDIFF(YEAR, birthdate, CURDATE()) >
    (SELECT AVG(TIMESTAMPDIFF(YEAR, birthdate, CURDATE())) FROM
customer);
```

51. Rank customers by spending (Window).

```
SELECT c.name,
    SUM(s.amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(s.amount) DESC) AS rank_no
FROM sales s
JOIN customer c ON s.customer_id = c.customer_id
GROUP BY c.name;
```

52. Running total of sales by date.

```
SELECT orderdate,
    SUM(amount) OVER (ORDER BY orderdate) AS running_total
FROM sales;
```

53. Monthly sales and month-over-month growth.

```
SELECT d.month,
       SUM(s.amount) AS total,
       SUM(SUM(s.amount)) OVER (ORDER BY d.month)
       - SUM(SUM(s.amount)) OVER (ORDER BY d.month ROWS BETWEEN 1
PRECEDING AND 1 PRECEDING)
       AS mom_growth
FROM sales s
JOIN date d ON s.orderdate = d.date
GROUP BY d.month;
```

54. Dense rank products by revenue.

```
SELECT p.prodname,
       SUM(s.amount) AS revenue,
       DENSE_RANK() OVER (ORDER BY SUM(s.amount) DESC) AS rev_rank
FROM sales s
JOIN product p ON s.product_id = p.product_id
GROUP BY p.prodname;
```

55. Top 3 vendors per country.

```
SELECT *
FROM (
    SELECT v.vendor_id,
           l.country,
           SUM(s.amount) AS revenue,
           DENSE_RANK() OVER
               (PARTITION BY l.country ORDER BY SUM(s.amount) DESC) AS rnk
    FROM sales s
    JOIN vendor v ON s.vendor_id = v.vendor_id
    JOIN customer c ON s.customer_id = c.customer_id
    JOIN location l ON c.zipcode = l.zipcode
    GROUP BY v.vendor_id, l.country
) t
WHERE rnk <= 3;
```

56. CTE: Highest selling product per month.

```
WITH monthly_sales AS (  
    SELECT p.prodname, d.month, SUM(s.amount) AS total  
    FROM sales s  
    JOIN product p ON s.product_id = p.product_id  
    JOIN date d ON s.orderdate = d.date  
    GROUP BY p.prodname, d.month  
)  
SELECT *  
FROM (  
    SELECT *,  
           RANK() OVER (PARTITION BY month ORDER BY total DESC) AS rnk  
    FROM monthly_sales  
) x  
WHERE rnk = 1;
```

57. CTE: Customers who purchased in consecutive years.

```
WITH years AS (  
    SELECT customer_id, YEAR(orderdate) AS yr  
    FROM sales  
    GROUP BY customer_id, YEAR(orderdate)  
)  
SELECT DISTINCT y1.customer_id  
FROM years y1  
JOIN years y2  
ON y1.customer_id = y2.customer_id  
AND y1.yr = y2.yr - 1;
```

59. Customers who bought from 3+ vendors.

```
SELECT customer_id, COUNT(DISTINCT vendor_id) AS vendor_count  
FROM sales  
GROUP BY customer_id  
HAVING COUNT(DISTINCT vendor_id) >= 3;
```